Department of Electronic & Telecommunication Engineering
University of Moratuwa, Sri Lanka

EN3160 - Image Processing and Machine Vision

Assignment 2 on Fitting and Alignment

Nadha M.I. 220409J

Date: 09.10.2025

Project link - Github

---

1. Blob Detection

The detected blobs are visualized as circles over the original image in **Figure 2**. To detect blobs of various sizes, a scale space was constructed by convolving the image with Gaussian filters at multiple scales (σ). Each σ value corresponds to a different level of smoothing; smaller σ detects fine details, while larger σ captures coarser, larger blobs. For each σ, the image was smoothed using a Gaussian filter and then processed with a Laplacian operator to highlight blob-like regions. The resulting responses were stacked into a 3D scale space (x, y, σ).

```python
#Scale-space parameters
sigma_values = np.linspace(1, 10, 10)
h, w = gray.shape
response_stack = np.zeros((h, w, len(sigma_values)))
# Laplacian of Gaussian (LoG) response for each σ
for i, sigma in enumerate(sigma_values):
    blurred = cv.GaussianBlur(gray, (0, 0), sigma)
    log_resp = cv.Laplacian(blurred, cv.CV_64F)
    response_stack[:, :, i] = np.abs(log_resp) * (sigma ** 2)
# Local maxima across space and scale
maxima = (response_stack == ndi.maximum_filter(response_stack, size=(3, 3, 3)))
```

Figure 1: Code snippet.

Figure 2: Detected blobs in the sunflower field.

Local maxima were detected across both spatial and scale dimensions using a $3 \times 3 \times 3$ neighborhood. Each voxel (x, y, σ) that is a local maximum represents a potential blob center at that scale. Coordinates of these maxima were extracted, and the corresponding radii were computed as $r = \sqrt{2}\sigma$
The top 30 strongest blob responses were selected and drawn as circles. This scale-space analysis enables the detection of flowers at varying distances and apparent sizes.

```
Largest Circle Parameters:
 Center: (0, 275)
 Radius: 9.90 pixels
 Corresponding σ: 7.00
 σ range used: 1.00 – 10.00
```

Figure 3: Largest Circle Parameters

The largest detected blob ($\sigma = 7$) corresponds to one of the larger sunflower heads in the image, representing a region with high blob response in the constructed scale-space.

2. Line and circle fitting

I used RANSAC to estimate a line and a circle from a noisy point set consisting of 50 line points with additive Gaussian noise ($s = 1.0$) and 50 circle points with small radial noise ($s \approx 0.625$). For the line, the distance threshold was set to 0.7, slightly below the noise, and the minimum inliers were set to 30 to ensure robust estimation while excluding outliers. For the circle, after removing the line inliers, the distance threshold was also set to 0.7 and the minimum inliers to 30, allowing the algorithm to accurately identify circle points despite remaining noise. These parameters were selected based on the expected noise levels and the proportion of points belonging to each structure, following standard RANSAC tuning heuristics.

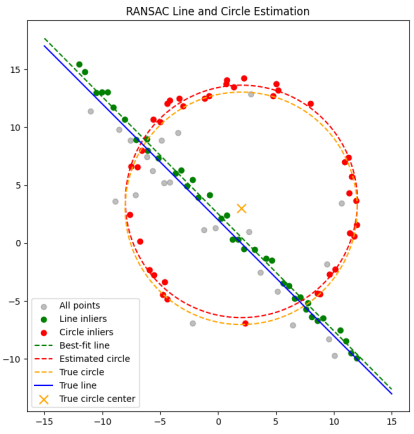
```python
def fit_line_normal_form(p1, p2):
    """Fit line in normal form: ax + by + d = 0, with ||[a,b]||=1."""
    (x1, y1), (x2, y2) = p1, p2
    dx, dy = x2 - x1, y2 - y1
    a, b = dy, -dx # Normal vector (perpendicular to direction)
    norm = np.sqrt(a**2 + b**2)
    a, b = a / norm, b / norm
    d = -(a * x1 + b * y1)
    return a, b, d
def point_line_distance(X, a, b, d):
    """Compute perpendicular distance from points to line."""
    return np.abs(a * X[:, 0] + b * X[:, 1] + d)
# RANSAC LLine implementation
def ransac_line(X, n_iterations=1000, dist_threshold=0.7, min_inliers=30):
    best_inliers = []
    best_model = None
    for _ in range(n_iterations):
        # Randomly choose 2 distinct points
        sample_idx = np.random.choice(len(X), 2, replace=False)
        p1, p2 = X[sample_idx]
        # Fit line model
        a, b, d = fit_line_normal_form(p1, p2)
        # Compute distances to all points
        distances = point_line_distance(X, a, b, d)
        # Determine inliers
        inliers = np.where(distances < dist_threshold)[0]
        # Update best model
        if len(inliers) > len(best_inliers) and len(inliers) > min_inliers:
            best_inliers = inliers
            best_model = (a, b, d)
    return best_model, best_inliers
```

(a) Line fitting using ransac

```python
# RANSAC for circle
def ransac_circle(X, n_iterations=1000, dist_threshold=0.7, min_inliers=25):
    best_inliers = []
    best_model = None
    N = len(X)
    for _ in range(n_iterations):
        sample_idx = np.random.choice(N, 3, replace=False)
        p1, p2, p3 = X[sample_idx]
        res = fit_circle_3pts(p1, p2, p3)
        if res is None:
            continue
        cx, cy, r = res
        errors = circle_radial_error(X, cx, cy, r)
        inliers = np.where(errors < dist_threshold)[0]
        if len(inliers) > len(best_inliers) and len(inliers) >= min_inliers:
            best_inliers = inliers
            best_model = (cx, cy, r)
    # refine using all inliers
    if best_model is not None and len(best_inliers) >= 3:
        def circle_error(params):
            cx, cy, r = params
            return np.sum((np.sqrt((X[best_inliers,0]-cx)**2 + (X[best_inliers,1]-cy)**2) - r)**2)
        res = minimize(circle_error, best_model)
        best_model = res.x

    return best_model, best_inliers
```

(b) Circle fitting using ransac



(c) RANSAC estimation of line and circle from a noisy point set

The results show that the line was estimated with parameters $a = -0.710$, $b = -0.704$, $d = 1.779$ and 38 inliers, while the circle was estimated with center $(2.018, 3.508)$ and radius 10.025, with 39 inliers. Both estimates are close to the ground truth values, demonstrating the robustness of RANSAC in separating the two structures and handling noisy measurements.

2.1. What will happen if we fit the circle first?

To illustrate the effect of fitting the circle first, I ran RANSAC on the full dataset without removing line points. The circle was estimated with center $(2.206, 3.531)$ and radius 10.019, with 41 inliers. The subsequent line estimate after removing these circle inliers was $a = -0.711$, $b = -0.703$, $d = 1.785$ with 33 inliers. Compared to the line-first approach, the circle estimate is slightly biased and the number of inliers for the line is reduced, **showing that fitting the line first is more robust when multiple structures coexist in the data**.

3. Planar Homography

```
h_flag, w_flag = flag_img.shape[:2]
pts_flag = np.array([[0, 0], [w_flag, 0], [w_flag, h_flag], [0, h_flag]], dtype=np.float32)

H, _ = cv2.findHomography(pts_flag, pts_arch)
warped_flag = cv2.warpPerspective(flag_img, H, (arch_img.shape[1], arch_img.shape[0]))
# Blend
flag_gray = cv2.cvtColor(warped_flag, cv2.COLOR_BGR2GRAY)
_, mask = cv2.threshold(flag_gray, 1, 255, cv2.THRESH_BINARY)
mask_inv = cv2.bitwise_not(mask)
arch_bg = cv2.bitwise_and(arch_img, arch_img, mask=mask_inv)
flag_fg = cv2.bitwise_and(warped_flag, warped_flag, mask=mask)
result = cv2.add(arch_bg, flag_fg)
```

The code shown is used to compute a homography between the four clicked points on the target image and the corners of the overlay image, apply a perspective warp using this homography, and blend the warped overlay onto the original image using a binary mask.

In the first example, I placed a Rapunzel image on a construction wall because the flat surface made the placement look natural and the colorful image stood out. In the second, I placed Kumar Sangakkara's picture on a cricket stadium screen, as the flat screen made the image clear and visually striking. Both choices show how images can be realistically overlaid on planar surfaces.

(a) Construction wall

(b) Rapunzel

(c) Superimposed image

Figure 5: Overlayed images on planar surfaces - example 1

(a) Stadium screen

(b) Sangakkara

(c) Superimposed image

Figure 6: Overlayed images on planar surfaces - example 2

4. Image Stitching

To stitch the graffiti images (`img1.ppm` onto `img5.ppm`), I first extracted SIFT features from both images. Two feature matching methods were compared: FLANN-based matcher and Brute-Force (BF) matcher. The BF matcher was selected due to its higher reliability, producing 87 good matches using Lowe's ratio test.

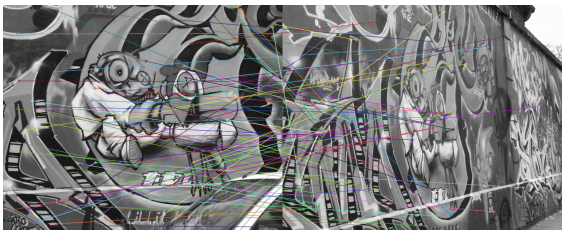


Figure 7: SIFT feature matching using BF matcher.

```python
def compute_homography(src_pts, dst_pts):
    """Compute homography from 4 points using DLT"""
    A = []
    for i in range(4):
        x, y = src_pts[i][0], src_pts[i][1]
        u, v = dst_pts[i][0], dst_pts[i][1]
        A.append([-x, -y, -1, 0, 0, 0, u*x, u*y, u])
        A.append([0, 0, 0, -x, -y, -1, v*x, v*y, v])
    A = np.array(A)
    _, _, Vt = np.linalg.svd(A)
    H = Vt[-1].reshape(3, 3)
    return H / H[2, 2]
def get_inliers(H, src_pts, dst_pts, threshold=3.0):
    """Compute inliers for homography"""
    pts = np.hstack([src_pts, np.ones((len(src_pts),1))])
    projected = (H @ pts.T).T
    projected /= projected[:,2][:, np.newaxis]
    dists = np.linalg.norm(projected[:, :2] - dst_pts, axis=1)
    inliers = np.where(dists < threshold)[0]
    return inliers
```

```python
def ransac_homography(src_pts, dst_pts, iterations=2000, threshold=5.0):
    best_H = None
    best_inliers = []
    n_points = 4
    for i in range(iterations):
        idx = np.random.choice(len(src_pts), n_points, replace=False)
        H = compute_homography(src_pts[idx], dst_pts[idx])
        inliers = get_inliers(H, src_pts, dst_pts, threshold)
        if len(inliers) > len(best_inliers):
            best_inliers = inliers
            best_H = H
    print(f"RANSAC inliers: {len(best_inliers)}/{len(src_pts)}")
    # Recompute H with all inliers
    if best_H is not None and len(best_inliers) > 4:
        best_H = compute_homography(src_pts[best_inliers[:4]], dst_pts[best_inliers[:4]])
    return best_H
```

Next, RANSAC was applied to estimate the homography and remove outliers. At each iteration, four correspondences were randomly chosen to compute a projective homography using the Direct Linear Transform (DLT). Points with a reprojection error below 5 pixels were counted as inliers, and after 2000 iterations, the homography with the maximum number of inliers was selected. The results were:

```
Good matches: 87
RANSAC inliers: 8/87
```

Figure 8: Stiched Image

Using the estimated homography, `img1.ppm` was warped onto the coordinate frame of `img5.ppm`. Overlapping regions were combined using mask-based blending to produce a seamless stitched image. Despite only 8 inliers being selected, the transformation accurately aligned the graffiti patterns, demonstrating the effectiveness of SIFT features, BF matching, and RANSAC-based homography estimation. Comparing the computed homography with the dataset's provided homography shows that while both achieve alignment in overlapping areas, differences arise from the sparse inliers and slight discrepancies in feature localization.